TP2 – Arbres

On créer une méthode taille(t) et hauteur(t) qui prend en argument un tableau sous forme d'arbre et renvoie le nombre de nœuds pour chacune des méthodes :

```python
def taille(self, t):
    if self.estVide(t):
        return 0
    else:
        taille_gauche=self.taille(t[1])
        taille_droite=self.taille(t[2])
        return 1+taille_gauche+taille_droite

def hauteur(self, t):
    if self.estVide(t):
        return -1
    else:
        hauteur_gauche=self.hauteur(t[1])
        hauteur_droite=self.hauteur(t[2])
        return 1+max(hauteur_gauche, hauteur_droite)
```

Puis une troisième méthode estVide(t) qui permet de déterminer si le tableau sous forme d'arbre pris en argument est vide et nous renvoie un booléen

```python
def estVide(self, t):
    if t==[] or t==None:
        return True
    else:
        return False
```

Voici donc la totalité de mon code de la classe arbre pour le moment :

```python
from PileFile import PileFile
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = (20, 6)

class arbre:

# à compléter
    def __init__(self):
        pass

    def taille(self, t):
        if self.estVide(t):
            return 0
        else:
            taille_gauche=self.taille(t[1])
            taille_droite=self.taille(t[2])
            return 1+taille_gauche+taille_droite

    def hauteur(self, t):
        if self.estVide(t):
            return -1
        else:
            hauteur_gauche=self.hauteur(t[1])
            hauteur_droite=self.hauteur(t[2])
            return 1+max(hauteur_gauche, hauteur_droite)

    def estVide(self, t):
        if t==[] or t==None:
            return True
        else:
            return False

    def dessiner_aux(self,t, rect, dy, labels, avl=False):
        if (self.estVide(t)):
            return
        x1, x2, y1, y2=rect
        xm= (x1+x2) //2
        x, t1, t2 = t
        self.dessiner_aux(t1, (x1, xm, y1, y2-dy), dy, labels, avl)
        self.dessiner_aux(t2, (xm, x2, y1, y2-dy), dy, labels, avl)
        if labels:
            if avl:
                plt.text(xm, y2, str(x[0]), fontsize=12, horizontalalignment='center')
            else:
                plt.text(xm, y2, str(x), fontsize=12, horizontalalignment='center')
        if not self.estVide(t1):
            a, b= ((xm, (x1+xm) //2), (y2, y2-dy))
            plt.plot(a, b, 'k', marker='o', markerfacecolor='c', markersize=25)
        if not self.estVide(t2):
            c, d= ((xm, (x2+xm) //2), (y2, y2-dy))
            plt.plot(c, d, 'k', marker='o', markerfacecolor='c', markersize=25)

    def dessiner(self,t, labels=True, avl=False):
        d=512
        pad=20
        dy= (d-2*pad) / (self.hauteur(t))
        self.dessiner_aux(t, (pad, d-pad, pad, d-pad), dy, labels, avl)
        plt.axis([0, d, 0, d])
        plt.axis('off')
        plt.show()
```

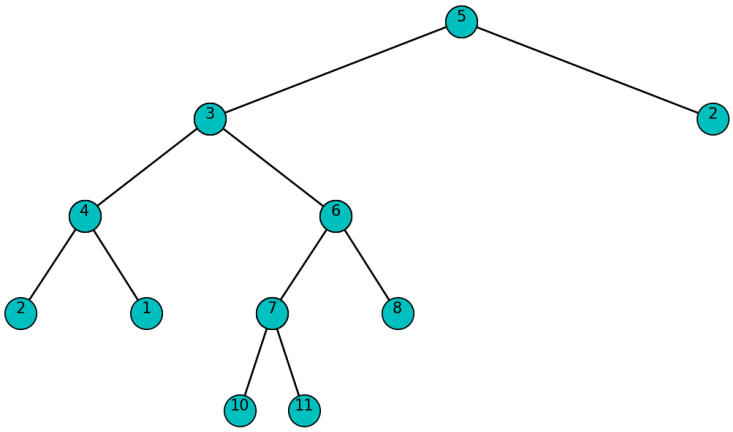
J'ai ensuite écrit mon main.py afin de réaliser des tests de mes méthodes ainsi que de voir la représentation de mes arbres pour chacun des exemples du TP et les autres arbres demandés :

```python
main.py > ...
from arbre_eleve import arbre

if __name__ == "__main__":
    a = [5,[3,[4,[2,[],[]],[1,[],[]]],[6,[7,[10,[],[]],[11,[],[]]],[8,[],[]]]],[2,[],[]]]
    b = [1,[2,[4,[8,[],[]],[9,[],[]]],[5,[10,[],[]],[]]],[3,[6,[],[]],[7,[],[]]]]
    c = [2, [1, [0, [], []], []], [5, [4, [3, [], []], []], [12,[9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]],[], [19, [], []]]]]]]]
    d = [20,[5,[3,[],[]],[12,[8, [6, [],[]],[]],[13,[],[]]]],[25, [21, [],[]],[28,[],[]]]]
    e = ['moi', ['maman', ['papy',[], []], ['mamie', [], []]], ['papa', ['papy',[], []], ['mamie', [], []]]]

liste_arbre = [a, b, c, d, e]
nom_arbre = ["a", "b", "c", "d", "e"]
arbre1 = arbre()

for i in range(len(nom_arbre)):
    print('Arbre ' + nom_arbre[i] + ' :')
    print('Taille : ' + str(arbre1.taille(liste_arbre[i])))
    print('Hauteur : ' + str(arbre1.hauteur(liste_arbre[i])))
    print('estVide : ' + str(arbre1.estVide(liste_arbre[i])))
    arbre1.dessiner(liste_arbre[i])
    print()
```

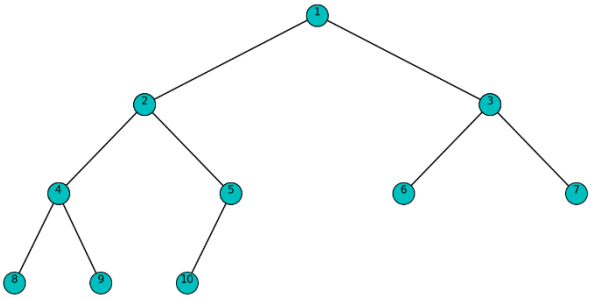
Voici les résultats obtenus dans le terminal puis les représentations affichées :

```
Arbre a :
Taille : 11
Hauteur : 4
estVide : False

Arbre b :
Taille : 10
Hauteur : 3
estVide : False

Arbre c :
Taille : 20
Hauteur : 8
estVide : False

Arbre d :
Taille : 10
Hauteur : 4
estVide : False

Arbre e :
Taille : 7
Hauteur : 2
estVide : False
```

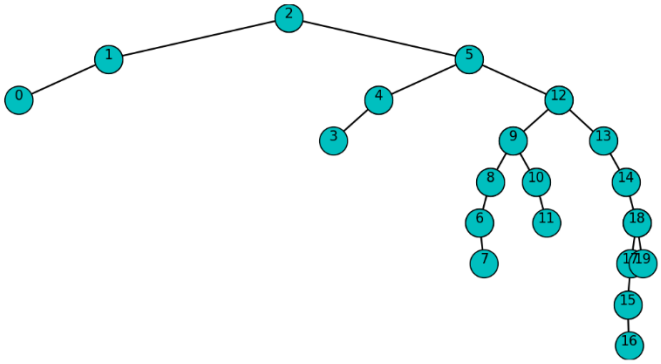
Représentation de a :



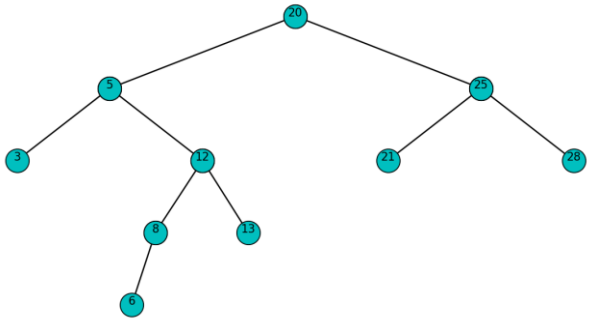
Représentation de b :



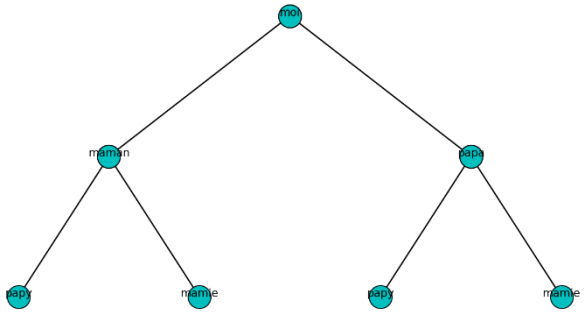
Représentation de c :



Représentation de d :



Représentation de e :



Nous avons programmé un algorithme de parcours en largeur des arbres à l'aide d'un objet PileFile de type File (FIFO) et qui affiche l'état de la pile à chaque étape du parcours. Voici la méthode programmée :

```python
def parcours_largeur(self,t):
    visite = PileFile()
    lst = []
    visite.enfiler(t)
    while visite.estVide() == False:
        visite.afficher()
        noeud = visite.defiler()
        if type(noeud) == list and noeud != []:
            lst.append(noeud[0])
            if noeud[1] != []:
                visite.enfiler(noeud[1])
            if noeud[2] != []:
                visite.enfiler(noeud[2])
        else:
            lst.append(noeud)
    return lst
```

Voici la main.py pour tester sur les tableaux proposés précédents :

```python
from arbre_eleve import arbre

if __name__ == "__main__":
    a = [5,[3,[4,[2,[],[]],[1,[],[]]],[6,[7,[10,[],[]],[11,[],[]]],[8,[],[]]]],[2,[],[]]]
    b = [1,[2,[4,[8,[],[]],[9,[],[]]],[5,[10,[],[]],[]]],[3,[6,[],[]],[7,[],[]]]]
    c = [2, [1, [0, [], []], []], [5, [4, [3, [], []], []], [12,[9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]],[]], [19, [], []]]]]]]]
    d = [20,[5,[3,[],[]],[12,[8, [6, [],[]],[]],[13,[],[]]]],[25, [21, [],[]],[28,[],[]]]]
    e = ['moi', ['maman', ['papy',[], []], ['mamie', [], []]], ['papa', ['papy',[], []], ['mamie', [], []]]]

liste_arbre = [a, b, c, d, e]
nom_arbre = ["a", "b", "c", "d", "e"]
arbre1 = arbre()

for i in range(len(nom_arbre)):
#    print('Arbre ' + nom_arbre[i] + ' :')
#    print('Taille : ' + str(arbre1.taille(liste_arbre[i])))
#    print('Hauteur : ' + str(arbre1.hauteur(liste_arbre[i])))
#    print('estVide : ' + str(arbre1.estVide(liste_arbre[i])))
#    print()
    print('Arbre ' + nom_arbre[i] + ' :')
    print('Parcours par largeur :' + str(arbre1.parcours_largeur(liste_arbre[i])))
    print()
```

Et voici les résultats obtenus dans la console :

```
Arbre a :
[[5, [3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]], [2, [], []]]]
[[3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]], [2, [], []]]
[[2, [], []], [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]]
[[4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]]
[[6, [7, [10, [], []], [11, [], []]], [8, [], []]], [2, [], []], [1, [], []]]
[[2, [], []], [1, [], []], [7, [10, [], []], [11, [], []]], [8, [], []]]
[[1, [], []], [7, [10, [], []], [11, [], []]], [8, [], []]]
[[7, [10, [], []], [11, [], []]], [8, [], []]]
[[8, [], []], [10, [], []], [11, [], []]]
[[10, [], []], [11, [], []]]
[[11, [], []]]
Parcours par largeur :[5, 3, 2, 4, 6, 2, 1, 7, 8, 10, 11]

Arbre b :
[[1, [2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]], [3, [6, [], []], [7, [], []]]]]
[[2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]], [3, [6, [], []], [7, [], []]]]
[[3, [6, [], []], [7, [], []]], [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]]
[[4, [8, [], []], [9, [], []]], [5, [10, [], []], []], [6, [], []], [7, [], []]]
[[5, [10, [], []], []], [6, [], []], [7, [], []], [8, [], []], [9, [], []]]
[[6, [], []], [7, [], []], [8, [], []], [9, [], []], [10, [], []]]
[[7, [], []], [8, [], []], [9, [], []], [10, [], []]]
[[8, [], []], [9, [], []], [10, [], []]]
[[9, [], []], [10, [], []]]
[[10, [], []]]
Parcours par largeur :[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
Arbre c :
[[2, [1, [0, [], []], []], [5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]]]]
[[1, [0, [], []], []], [5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]]]
[[5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]], [0, [], []]]
[[0, [], []], [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]]]
[[4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]]]
[[12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]], [3, [], []]]
[[3, [], []], [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]]]
[[9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]]]
[[13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]], [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]]
[[8, [6, [], [7, [], []]], []], [10, [], [11, [], []]], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]]]
[[10, [], [11, [], []]], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]], [6, [], [7, [], []]]]
[[14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]], [6, [], [7, [], []]], [11, [], []]]
[[6, [], [7, [], []]], [11, [], []], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]
[[11, [], []], [18, [17, [15, [], [16, [], []]], []], [19, [], []]], [7, [], []]]
[[18, [17, [15, [], [16, [], []]], []], [19, [], []]], [7, [], []]]
[[7, [], []], [17, [15, [], [16, [], []]], []], [19, [], []]]
[[17, [15, [], [16, [], []]], []], [19, [], []]]
[[19, [], []], [15, [], [16, [], []]]]
[[15, [], [16, [], []]]]
[[16, [], []]]
Parcours par largeur :[2, 1, 5, 0, 4, 12, 3, 9, 13, 8, 10, 14, 6, 11, 18, 7, 17, 19, 15, 16]

Arbre d :
[[20, [5, [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]], [25, [21, [], []], [28, [], []]]]]
[[5, [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]], [25, [21, [], []], [28, [], []]]]
[[25, [21, [], []], [28, [], []]], [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]]
[[3, [], []], [12, [8, [6, [], []], []], [13, [], []]], [21, [], []], [28, [], []]]
[[12, [8, [6, [], []], []], [13, [], []]], [21, [], []], [28, [], []]]
[[21, [], []], [28, [], []], [8, [6, [], []], []], [13, [], []]]
[[28, [], []], [8, [6, [], []], []], [13, [], []]]
[[8, [6, [], []], []], [13, [], []]]
[[13, [], []], [6, [], []]]
[[6, [], []]]
Parcours par largeur :[20, 5, 25, 3, 12, 21, 28, 8, 13, 6]
```

```
Arbre e :
[['moi', ['maman', ['papy', [], []], ['mamie', [], []]], ['papa', ['papy', [], []], ['mamie', [], []]]]]
[['maman', ['papy', [], []], ['mamie', [], []]], ['papa', ['papy', [], []], ['mamie', [], []]]]
[['papa', ['papy', [], []], ['mamie', [], []]], ['papy', [], []], ['mamie', [], []]]
[['papy', [], []], ['mamie', [], []], ['papy', [], []], ['mamie', [], []]]
[['mamie', [], []], ['papy', [], []], ['mamie', [], []]]
[['papy', [], []], ['mamie', [], []]]
[['mamie', [], []]]
Parcours par largeur :['moi', 'maman', 'papa', 'papy', 'mamie', 'papy', 'mamie']
```

Nous avons programmé un algorithme de parcours en profondeur des arbres à l'aide d'un objet PileFile de type Pile (LIFO) et qui affiche l'état de la pile à chaque étape du parcours. Voici la méthode programmée :

```python
def parcours_profondeur(self,t):
    visite = PileFile()
    lst = []
    visite.empiler(t)
    while visite.estVide() == False:
        visite.afficher()
        noeud = visite.depiler()
        if type(noeud) == list and noeud != []:
            lst.append(noeud[0])
            if noeud[2] != []:
                visite.empiler(noeud[2])
            if noeud[1] != []:
                visite.empiler(noeud[1])
        else:
            lst.append(noeud)
    return lst
```

Voici la main.py pour tester sur les tableaux proposés précédents :

```python
from arbre_eleve import arbre

if __name__ == "__main__":
    a = [5,[3,[4,[2,[],[]],[1,[],[]]],[6,[7,[10,[],[]],[11,[],[]]],[8,[],[]]]],[2,[],[]]]
    b = [1,[2,[4,[8,[],[]],[9,[],[]]],[5,[10,[],[]],[]]],[3,[6,[],[]],[7,[],[]]]]
    c = [2, [1, [0, [], []], []], [5, [4, [3, [], []], []], [12,[9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]],[]], [19, [], []]]]]]]]
    d = [20,[5,[3,[],[]],[12,[8, [6, [],[]],[]],[13,[],[]]]],[25, [21, [],[]],[28,[],[]]]]
    e = ['moi', ['maman', ['papy',[], []], ['mamie', [], []]], ['papa', ['papy',[], []], ['mamie', [], []]]]

liste_arbre = [a, b, c, d, e]
nom_arbre = ["a", "b", "c", "d", "e"]
arbre1 = arbre()

for i in range(len(nom_arbre)):
#    print('Arbre ' + nom_arbre[i] + ' :')
#    print('Taille : ' + str(arbre1.taille(liste_arbre[i])))
#    print('Hauteur : ' + str(arbre1.hauteur(liste_arbre[i])))
#    print('estVide : ' + str(arbre1.estVide(liste_arbre[i])))
#    print()
    print('Arbre ' + nom_arbre[i] + ' :')
#    print('Parcours par largeur :' + str(arbre1.parcours_largeur(liste_arbre[i])))
    print('Parcours par profondeur :' + str(arbre1.parcours_profondeur(liste_arbre[i])))
    print()
```

Et voici les résultats obtenus dans la console :

```
Arbre a :
[[5, [3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]], [2, [], []]]]
[[2, [], []], [3, [4, [2, [], []], [1, [], []]], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]]]
[[2, [], []], [6, [7, [10, [], []], [11, [], []]], [8, [], []]], [4, [2, [], []], [1, [], []]]]
[[2, [], []], [6, [7, [10, [], []], [11, [], []]], [8, [], []]], [1, [], []], [2, [], []]]
[[2, [], []], [6, [7, [10, [], []], [11, [], []]], [8, [], []]], [1, [], []]]
[[2, [], []], [6, [7, [10, [], []], [11, [], []]], [8, [], []]]]
[[2, [], []], [8, [], []], [7, [10, [], []], [11, [], []]]]
[[2, [], []], [8, [], []], [11, [], []], [10, [], []]]
[[2, [], []], [8, [], []], [11, [], []]]
[[2, [], []], [8, [], []]]
[[2, [], []]]
Parcours par profondeur :[5, 3, 4, 2, 1, 6, 7, 10, 11, 8, 2]

Arbre b :
[[1, [2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]], [3, [6, [], []], [7, [], []]]]]
[[3, [6, [], []], [7, [], []]], [2, [4, [8, [], []], [9, [], []]], [5, [10, [], []], []]]]
[[3, [6, [], []], [7, [], []]], [5, [10, [], []], []], [4, [8, [], []], [9, [], []]]]
[[3, [6, [], []], [7, [], []]], [5, [10, [], []], []], [9, [], []], [8, [], []]]
[[3, [6, [], []], [7, [], []]], [5, [10, [], []], []], [9, [], []]]
[[3, [6, [], []], [7, [], []]], [5, [10, [], []], []]]
[[3, [6, [], []], [7, [], []]], [10, [], []]]
[[3, [6, [], []], [7, [], []]]]
[[7, [], []], [6, [], []]]
[[7, [], []]]
Parcours par profondeur :[1, 2, 4, 8, 9, 5, 10, 3, 6, 7]
```

```
Arbre c :
[[2, [1, [0, [], []], []], [5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]]]]
[[5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]], [1, [0, [], []], []]]
[[5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]], [0, [], []]]
[[5, [4, [3, [], []], []], [12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]]]
[[12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]], [4, [3, [], []], []]]
[[12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]], [3, [], []]]
[[12, [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]]]
[[13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]], [9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]]]
[[13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]], [10, [], [11, [], []]], [8, [6, [], [7, [], []]], []]]
[[13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]], [10, [], [11, [], []]], [6, [], [7, [], []]]]
[[13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]], [10, [], [11, [], []]], [7, [], []]]
[[13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]], [10, [], [11, [], []]]]
[[13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]], [11, [], []]]
[[13, [], [14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]
[[14, [], [18, [17, [15, [], [16, [], []]], []], [19, [], []]]]]
[[18, [17, [15, [], [16, [], []]], []], [19, [], []]]]
[[19, [], []], [17, [15, [], [16, [], []]], []]]
[[19, [], []], [15, [], [16, [], []]]]
[[19, [], []], [16, [], []]]
[[19, [], []]]
Parcours par profondeur :[2, 1, 0, 5, 4, 3, 12, 9, 8, 6, 7, 10, 11, 13, 14, 18, 17, 15, 16, 19]

Arbre d :
[[20, [5, [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]], [25, [21, [], []], [28, [], []]]]]
[[25, [21, [], []], [28, [], []]], [5, [3, [], []], [12, [8, [6, [], []], []], [13, [], []]]]]
[[25, [21, [], []], [28, [], []]], [12, [8, [6, [], []], []], [13, [], []]], [3, [], []]]
[[25, [21, [], []], [28, [], []]], [12, [8, [6, [], []], []], [13, [], []]]]
[[25, [21, [], []], [28, [], []]], [13, [], []], [8, [6, [], []], []]]
[[25, [21, [], []], [28, [], []]], [13, [], []], [6, [], []]]
[[25, [21, [], []], [28, [], []]], [13, [], []]]
[[25, [21, [], []], [28, [], []]]]
[[28, [], []], [21, [], []]]
[[28, [], []]]
Parcours par profondeur :[20, 5, 3, 12, 8, 6, 13, 25, 21, 28]
```

```
Arbre e :
[['moi', ['maman', ['papy', [], []], ['mamie', [], []]], ['papa', ['papy', [], []], ['mamie', [], []]]]]
[['papa', ['papy', [], []], ['mamie', [], []]], ['maman', ['papy', [], []], ['mamie', [], []]]]
[['papa', ['papy', [], []], ['mamie', [], []]], ['mamie', [], []], ['papy', [], []]]
[['papa', ['papy', [], []], ['mamie', [], []]], ['mamie', [], []]]
[['papa', ['papy', [], []], ['mamie', [], []]]]
[['mamie', [], []], ['papy', [], []]]
[['mamie', [], []]]
Parcours par profondeur :['moi', 'maman', 'papy', 'mamie', 'papa', 'papy', 'mamie']
```

Nous avons programmé un algorithme de parcours en profondeur de type préfixe et récursif des arbres à l'aide d'un objet PileFile de type Pile (LIFO) et qui affiche l'état de la pile à chaque étape du parcours. Voici la méthode programmée :

```python
def parcours_profondeur_recursif_prefix(self, t):
    lst = []
    if type(t) == list and t != []:
        lst.append(t[0])
        print(lst)
        if t[1] != []:
            res1 = self.parcours_profondeur_recursif_prefix(t[1])
            lst.append(res1)
            print(lst)
        if t[2] != []:
            res2 = self.parcours_profondeur_recursif_prefix(t[2])
            lst.append(res2)
            print(lst)
    else:
        lst.append(t)
        print(lst)
    return lst
```

Voici la main.py pour tester sur les tableaux proposés précédents :

```python
main.py > ...
from arbre_eleve import arbre

if __name__ == "__main__":
    a = [5,[3,[4,[2,[],[]],[1,[],[]]],[6,[7,[10,[],[]],[11,[],[]]],[8,[],[]]]],[2,[],[]]]
    b = [1,[2,[4,[8,[],[]],[9,[],[]]],[5,[10,[],[]],[]]],[3,[6,[],[]],[7,[],[]]]]
    c = [2, [1, [0, [], []], []], [], [5, [4, [3, [], [], [], [], [12,[9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]],[]], [19, [], []]]]]]]]]
    d = [20,[5,[3,[],[]],[12,[8, [6, [],[]],[]],[13,[],[]]]],[25, [21, [],[]],[28,[],[]]]]
    e = ['moi', ['maman', ['papy',[], []], ['mamie', [], []]], ['papa', ['papy',[], []], ['mamie', [], []]]]

liste_arbre = [a, b, c, d, e]
nom_arbre = ["a", "b", "c", "d", "e"]
arbre1 = arbre()

for i in range(len(nom_arbre)):
#    print('Arbre ' + nom_arbre[i] + ' :')
#    print('Taille : ' + str(arbre1.taille(liste_arbre[i])))
#    print('Hauteur : ' + str(arbre1.hauteur(liste_arbre[i])))
#    print('estVide : ' + str(arbre1.estVide(liste_arbre[i])))
#    print()
    print('Arbre ' + nom_arbre[i] + ' :')
#    print('Parcours par largeur :' + str(arbre1.parcours_largeur(liste_arbre[i])))
#    print('Parcours par profondeur :' + str(arbre1.parcours_profondeur(liste_arbre[i])))
    print('Parcours par profondeur récursif prefix :' + str(arbre1.parcours_profondeur_recursif_prefix(liste_arbre[i])))
    print()
```

Et voici les résultats obtenus dans la console :

```
Arbre a :
[5]
[3]
[4]
[2]
[4, [2]]
[1]
[4, [2], [1]]
[3, [4, [2], [1]]]
[6]
[7]
[10]
[7, [10]]
[11]
[7, [10], [11]]
[6, [7, [10], [11]]]
[8]
[6, [7, [10], [11]], [8]]
[3, [4, [2], [1]], [6, [7, [10], [11]], [8]]]
[5, [3, [4, [2], [1]], [6, [7, [10], [11]], [8]]]]
[2]
[5, [3, [4, [2], [1]], [6, [7, [10], [11]], [8]]], [2]]
Parcours par profondeur récursif prefix :[5, [3, [4, [2], [1]], [6, [7, [10], [11]], [8]]], [2]]
```

```
Arbre b :
[1]
[2]
[4]
[8]
[4, [8]]
[9]
[4, [8], [9]]
[2, [4, [8], [9]]]
[5]
[10]
[5, [10]]
[2, [4, [8], [9]], [5, [10]]]
[1, [2, [4, [8], [9]], [5, [10]]]]
[3]
[6]
[3, [6]]
[7]
[3, [6], [7]]
[1, [2, [4, [8], [9]], [5, [10]]], [3, [6], [7]]]
Parcours par profondeur récursif prefix :[1, [2, [4, [8], [9]], [5, [10]]], [3, [6], [7]]]
```

```
Arbre c :
[2]
[1]
[0]
[1, [0]]
[2, [1, [0]]]
[5]
[4]
[3]
[4, [3]]
[5, [4, [3]]]
[12]
[9]
[8]
[6]
[7]
[6, [7]]
[8, [6, [7]]]
[9, [8, [6, [7]]]]
[10]
[11]
[10, [11]]
[9, [8, [6, [7]]], [10, [11]]]
[12, [9, [8, [6, [7]]], [10, [11]]]]
[13]
[14]
[18]
[17]
[15]
[16]
[15, [16]]
[17, [15, [16]]]
[18, [17, [15, [16]]]]
[19]
[18, [17, [15, [16]]], [19]]
[14, [18, [17, [15, [16]]], [19]]]
[13, [14, [18, [17, [15, [16]]], [19]]]]
[12, [9, [8, [6, [7]]], [10, [11]]], [13, [14, [18, [17, [15, [16]]], [19]]]]]
[5, [4, [3]], [12, [9, [8, [6, [7]]], [10, [11]]], [13, [14, [18, [17, [15, [16]]], [19]]]]]]
[2, [1, [0]], [5, [4, [3]], [12, [9, [8, [6, [7]]], [10, [11]]], [13, [14, [18, [17, [15, [16]]], [19]]]]]]]
Parcours par profondeur récursif prefix :[2, [1, [0]], [5, [4, [3]], [12, [9, [8, [6, [7]]], [10, [11]]], [13, [14, [18, [17, [15, [16]]], [19]]]]]]]
```

```
Arbre d :
[20]
[5]
[3]
[5, [3]]
[12]
[8]
[6]
[8, [6]]
[3]
[5, [3]]
[12]
[8]
[6]
[8, [6]]
[12, [8, [6]]]
[8, [6]]
[12, [8, [6]]]
[13]
[12, [8, [6]], [13]]
[5, [3], [12, [8, [6]], [13]]]
[20, [5, [3], [12, [8, [6]], [13]]]]
[25]
[21]
[25, [21]]
[28]
[25, [21], [28]]
[20, [5, [3], [12, [8, [6]], [13]]], [25, [21], [28]]]
Parcours par profondeur récursif prefix :[20, [5, [3], [12, [8, [6]], [13]]], [25, [21], [28]]]
```

```
Arbre e :
['moi']
['maman']
['papy']
['maman', ['papy']]
['mamie']
['maman', ['papy'], ['mamie']]
['moi', ['maman', ['papy'], ['mamie']]]
['papa']
['papy']
['papa', ['papy']]
['mamie']
['papa', ['papy'], ['mamie']]
['moi', ['maman', ['papy'], ['mamie']], ['papa', ['papy'], ['mamie']]]
Parcours par profondeur récursif prefix :['moi', ['maman', ['papy'], ['mamie']], ['papa', ['papy'], ['mamie']]]
```

Nous avons programmé un algorithme de parcours en profondeur de type infixe et récursif des arbres à l'aide d'un objet PileFile de type Pile (LIFO) et qui affiche l'état de la pile à chaque étape du parcours. Voici la méthode programmée :

```python
def parcours_profondeur_recursif_infixe(self, t):
    lst = []
    if type(t) == list and t != []:
        if t[1] != []:
            res1 = self.parcours_profondeur_recursif_infixe(t[1])
            lst.append(res1)
            print(lst)
        lst.append(t[0])
        print(lst)
        if t[2] != []:
            res2 = self.parcours_profondeur_recursif_infixe(t[2])
            lst.append(res2)
            print(lst)
    else:
        lst.append(t)
        print(lst)
    return lst
```

Voici la main.py pour tester sur les tableaux proposés précédents :

```python
main.py > ...
from arbre_eleve import arbre

if __name__ == "__main__":
    a = [5,[3,[4,[2,[],[]],[1,[],[]]],[6,[7,[10,[],[]],[11,[],[]]],[8,[],[]]]],[2,[],[]]]
    b = [1,[2,[4,[8,[],[]],[9,[],[]]],[5,[10,[],[]],[]]],[3,[6,[],[]],[7,[],[]]]]
    c = [2, [1, [0, [], []], []], [5, [4, [3, [], []], []], [12,[9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]],[]], [19, [], []]]]]]]
    d = [20,[5,[3,[],[]],[12,[8, [6, [],[]],[]],[13,[],[]]]],[25, [21, [],[]],[28,[],[]]]]
    e = ['moi', ['maman', ['papy',[], []], ['mamie', [], []]], ['papa', ['papy',[], []], ['mamie', [], []]]]

liste_arbre = [a, b, c, d, e]
nom_arbre = ["a", "b", "c", "d", "e"]
arbre1 = arbre()

for i in range(len(nom_arbre)):
#    print('Arbre ' + nom_arbre[i] + ' :')
#    print('Taille : ' + str(arbre1.taille(liste_arbre[i])))
#    print('Hauteur : ' + str(arbre1.hauteur(liste_arbre[i])))
#    print('estVide : ' + str(arbre1.estVide(liste_arbre[i])))
#    print()
    print('Arbre ' + nom_arbre[i] + ' :')
#    print('Parcours par largeur :' + str(arbre1.parcours_largeur(liste_arbre[i])))
#    print('Parcours par profondeur :' + str(arbre1.parcours_profondeur(liste_arbre[i])))
#    print('Parcours par profondeur récursif prefix :' + str(arbre1.parcours_profondeur_recursif_prefix(liste_arbre[i])))
    print('Parcours par profondeur récursif infixe :' + str(arbre1.parcours_profondeur_recursif_infixe(liste_arbre[i])))
    print()
```

Et voici les résultats obtenus dans la console :

```
Arbre a :
[2]
[[2]]
[[2], 4]
[1]
[[2], 4, [1]]
[[[2], 4, [1]]]
[[[2], 4, [1]], 3]
[10]
[[10]]
[[10], 7]
[11]
[[10], 7, [11]]
[[[10], 7, [11]]]
[[[10], 7, [11]], 6]
[8]
[[[10], 7, [11]], 6, [8]]
[[[2], 4, [1]], 3, [[[10], 7, [11]], 6, [8]]]
[[[[2], 4, [1]], 3, [[[10], 7, [11]], 6, [8]]]]
[[[[2], 4, [1]], 3, [[[10], 7, [11]], 6, [8]]], 5]
[2]
[[[[2], 4, [1]], 3, [[[10], 7, [11]], 6, [8]]], 5, [2]]
Parcours par profondeur récursif infixe :[[[[2], 4, [1]], 3, [[[10], 7, [11]], 6, [8]]], 5, [2]]

Arbre b :
[8]
[[8]]
[[8], 4]
[9]
[[8], 4, [9]]
[[[8], 4, [9]]]
[[[8], 4, [9]], 2]
[10]
[[10]]
[[10], 5]
[[[8], 4, [9]], 2, [[10], 5]]
[[[[8], 4, [9]], 2, [[10], 5]]]
[[[[8], 4, [9]], 2, [[10], 5]], 1]
[6]
[[6]]
[[6], 3]
[7]
[[6], 3, [7]]
[[[[8], 4, [9]], 2, [[10], 5]], 1, [[6], 3, [7]]]
Parcours par profondeur récursif infixe :[[[[8], 4, [9]], 2, [[10], 5]], 1, [[6], 3, [7]]]
```

```
Arbre c :
[0]
[[0]]
[[0], 1]
[[[0], 1]]
[[[0], 1], 2]
[3]
[[3]]
[[3], 4]
[[[3], 4]]
[[[3], 4], 5]
[6]
[7]
[6, [7]]
[[6, [7]]]
[[6, [7]], 8]
[[[6, [7]], 8]]
[[[6, [7]], 8], 9]
[10]
[11]
[10, [11]]
[[[6, [7]], 8], 9, [10, [11]]]
[[[[6, [7]], 8], 9, [10, [11]]]]
[[[[6, [7]], 8], 9, [10, [11]]], 12]
[13]
[14]
[15]
[16]
[15, [16]]
[[15, [16]]]
[[15, [16]], 17]
[[[15, [16]], 17]]
[[[15, [16]], 17], 18]
[19]
[[[15, [16]], 17], 18, [19]]
[14, [[[15, [16]], 17], 18, [19]]]
[13, [14, [[[15, [16]], 17], 18, [19]]]]
[[[[6, [7]], 8], 9, [10, [11]]], 12, [13, [14, [[[15, [16]], 17], 18, [19]]]]]
[[[3], 4], 5, [[[[6, [7]], 8], 9, [10, [11]]], 12, [13, [14, [[[15, [16]], 17], 18, [19]]]]]]
[[[0], 1], 2, [[[3], 4], 5, [[[[6, [7]], 8], 9, [10, [11]]], 12, [13, [14, [[[15, [16]], 17], 18, [19]]]]]]]
Parcours par profondeur récursif infixe :[[[0], 1], 2, [[[3], 4], 5, [[[[6, [7]], 8], 9, [10, [11]]], 12, [13, [14, [[[15, [16]], 17], 18, [19]]]]]]]
```

```
Arbre d :
[3]
[[3]]
[[3], 5]
[6]
[[6]]
[[6], 8]
[[[6], 8]]
[[[6], 8], 12]
[13]
[[[6], 8], 12, [13]]
[[3], 5, [[[6], 8], 12, [13]]]
[[[3], 5, [[[6], 8], 12, [13]]]]
[[[3], 5, [[[6], 8], 12, [13]]], 20]
[21]
[[21]]
[[21], 25]
[28]
[[21], 25, [28]]
[[[3], 5, [[[6], 8], 12, [13]]], 20, [[21], 25, [28]]]
Parcours par profondeur récursif infixe :[[[3], 5, [[[6], 8], 12, [13]]], 20, [[21], 25, [28]]]

Arbre e :
['papy']
[['papy']]
[['papy'], 'maman']
['mamie']
[['papy'], 'maman', ['mamie']]
[[['papy'], 'maman', ['mamie']]]
[[['papy'], 'maman', ['mamie']], 'moi']
['papy']
[['papy']]
[['papy'], 'papa']
['mamie']
[['papy'], 'papa', ['mamie']]
[[['papy'], 'maman', ['mamie']], 'moi', [['papy'], 'papa', ['mamie']]]
Parcours par profondeur récursif infixe :[[['papy'], 'maman', ['mamie']], 'moi', [['papy'], 'papa', ['mamie']]]
```

Nous avons programmé un algorithme de parcours en profondeur de type postfixe et récursif des arbres à l'aide d'un objet PileFile de type Pile (LIFO) et qui affiche l'état de la pile à chaque étape du parcours. Voici la méthode programmée :

```python
def parcours_profondeur_recursif_postfixe(self, t):
    lst = []
    if type(t) == list and t != []:
        if t[1] != []:
            res1 = self.parcours_profondeur_recursif_postfixe(t[1])
            lst.append(res1)
            print(lst)
        if t[2] != []:
            res2 = self.parcours_profondeur_recursif_postfixe(t[2])
            lst.append(res2)
            print(lst)
        lst.append(t[0])
    else:
        lst.append(t)
        print(lst)
    return lst
```

Voici la main.py pour tester sur les tableaux proposés précédents :

```python
main.py > ...
from arbre_eleve import arbre

if __name__ == "__main__":
    a = [5,[3,[4,[2,[],[]],[1,[],[]]],[6,[7,[10,[],[]],[11,[],[]]],[8,[],[]]]],[2,[],[]]]
    b = [1,[2,[4,[8,[],[]],[9,[],[]]],[5,[10,[],[]],[]]],[3,[6,[],[]],[7,[],[]]]]
    c = [2, [1, [0, [], []], []], [5, [4, [3, [], []], []], [12,[9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]],[]], [19, [], []]]]]]]]
    d = [20,[5,[3,[],[]],[12,[8, [6, [],[]],[]],[13,[],[]]]],[25, [21, [],[]],[28,[],[]]]]
    e = ['moi', ['maman', ['papy',[], []], ['mamie', [], []]], ['papa', ['papy',[], []], ['mamie', [], []]]]

liste_arbre = [a, b, c, d, e]
nom_arbre = ["a", "b", "c", "d", "e"]
arbre1 = arbre()

for i in range(len(nom_arbre)):
#    print('Arbre ' + nom_arbre[i] + ' :')
#    print('Taille : ' + str(arbre1.taille(liste_arbre[i])))
#    print('Hauteur : ' + str(arbre1.hauteur(liste_arbre[i])))
#    print('estVide : ' + str(arbre1.estVide(liste_arbre[i])))
#    print()
    print('Arbre ' + nom_arbre[i] + ' :')
#    print('Parcours par largeur :' + str(arbre1.parcours_largeur(liste_arbre[i])))
#    print('Parcours par profondeur :' + str(arbre1.parcours_profondeur(liste_arbre[i])))
#    print('Parcours par profondeur récursif prefix :' + str(arbre1.parcours_profondeur_recursif_prefix(liste_arbre[i])))
#    print('Parcours par profondeur récursif infixe :' + str(arbre1.parcours_profondeur_recursif_infixe(liste_arbre[i])))
    print('Parcours par profondeur récursif postfixe :' + str(arbre1.parcours_profondeur_recursif_postfixe(liste_arbre[i])))
    print()
```

Et voici les résultats obtenus dans la console :

```
Arbre a :
[[2]]
[[2], [1]]
[[[2], [1], 4]]
[[10]]
[[10], [11]]
[[[10], [11], 7]]
[[[10], [11], 7], [8]]
[[[2], [1], 4], [[[10], [11], 7], [8], 6]]
[[[[2], [1], 4], [[[10], [11], 7], [8], 6], 3]]
[[[[2], [1], 4], [[[10], [11], 7], [8], 6], 3], [2]]
Parcours par profondeur récursif postfixe :[[[[2], [1], 4], [[[10], [11], 7], [8], 6], 3], [2], 5]

Arbre b :
[[8]]
[[8], [9]]
[[[8], [9], 4]]
[[10]]
[[[8], [9], 4], [[10], 5]]
[[[[8], [9], 4], [[10], 5], 2]]
[[6]]
[[6], [7]]
[[[[8], [9], 4], [[10], 5], 2], [[6], [7], 3]]
Parcours par profondeur récursif postfixe :[[[[8], [9], 4], [[10], 5], 2], [[6], [7], 3], 1]

Arbre c :
[[0]]
[[[0], 1]]
[[3]]
[[[3], 4]]
[[7]]
[[[7], 6]]
[[[[7], 6], 8]]
[[11]]
[[[[7], 6], 8], [[11], 10]]
[[[[[7], 6], 8], [[11], 10], 9]]
[[16]]
[[[16], 15]]
[[[[16], 15], 17]]
[[[[16], 15], 17], [19]]
[[[[[16], 15], 17], [19], 18]]
[[[[[[16], 15], 17], [19], 18], 14]]
[[[[[7], 6], 8], [[11], 10], 9], [[[[[[16], 15], 17], [19], 18], 14], 13]]
[[[3], 4], [[[[[7], 6], 8], [[11], 10], 9], [[[[[[16], 15], 17], [19], 18], 14], 13], 12]]
[[[0], 1], [[[3], 4], [[[[[7], 6], 8], [[11], 10], 9], [[[[[[16], 15], 17], [19], 18], 14], 13], 12], 5]]
Parcours par profondeur récursif postfixe :[[[0], 1], [[[3], 4], [[[[[7], 6], 8], [[11], 10], 9], [[[[[[16], 15], 17], [19], 18], 14], 13], 12], 5], 2]
```

```
Arbre d :
[[3]]
[[6]]
[[[6], 8]]
[[[6], 8], [13]]
[[3], [[[6], 8], [13], 12]]
[[[3], [[[6], 8], [13], 12], 5]]
[[21]]
[[21], [28]]
[[[3], [[[6], 8], [13], 12], 5], [[21], [28], 25]]
Parcours par profondeur récursif postfixe :[[[3], [[[6], 8], [13], 12], 5], [[21], [28], 25], 20]

Arbre e :
[['papy']]
[['papy'], ['mamie']]
[[['papy'], ['mamie'], 'maman']]
[['papy']]
[['papy'], ['mamie']]
[[['papy'], ['mamie'], 'maman'], [['papy'], ['mamie'], 'papa']]
Parcours par profondeur récursif postfixe :[[['papy'], ['mamie'], 'maman'], [['papy'], ['mamie'], 'papa'], 'moi']
```

Voici donc pour finir la totalité du code :

Main.py :

```python
main.py > ...
from arbre_eleve import arbre

if __name__ == "__main__":
    a = [5,[3,[4,[2,[],[]],[1,[],[]]],[6,[7,[10,[],[]],[11,[],[]]],[8,[],[]]]],[2,[],[]]]
    b = [1,[2,[4,[8,[],[]],[9,[],[]]],[5,[10,[],[]],[]]],[3,[6,[],[]],[7,[],[]]]]
    c = [2, [1, [0, [], []], []], [], [5, [4, [3, [], []], []], [12,[9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]],[]], [19, [], []]]]]]]]
    d = [20,[5,[3,[],[]],[12,[8, [6, [],[]],[]],[13,[],[]]]],[25, [21, [],[]],[28,[],[]]]]
    e = ['moi', ['maman', ['papy',[], []], ['mamie', [], []]], ['papa', ['papy',[], []], ['mamie', [], []]]]

liste_arbre = [a, b, c, d, e]
nom_arbre = ["a", "b", "c", "d", "e"]
arbre1 = arbre()

for i in range(len(nom_arbre)):
    print('Arbre ' + nom_arbre[i] + ' :')
    print('Taille : ' + str(arbre1.taille(liste_arbre[i])))
    print('Hauteur : ' + str(arbre1.hauteur(liste_arbre[i])))
    print('estVide : ' + str(arbre1.estVide(liste_arbre[i])))
    print()
    print('Arbre ' + nom_arbre[i] + ' :')
    print('Parcours par largeur :' + str(arbre1.parcours_largeur(liste_arbre[i])))
    print('Parcours par profondeur :' + str(arbre1.parcours_profondeur(liste_arbre[i])))
    print('Parcours par profondeur récursif prefix :' + str(arbre1.parcours_profondeur_recursif_prefix(liste_arbre[i])))
    print('Parcours par profondeur récursif infixe :' + str(arbre1.parcours_profondeur_recursif_infixe(liste_arbre[i])))
    print('Parcours par profondeur récursif postfixe :' + str(arbre1.parcours_profondeur_recursif_postfixe(liste_arbre[i])))
    print()
```

Arbre_eleve.py :

arbre_eleve.py > arbre > parcours_profondeur_recursif_infixe

```python
from PileFile import PileFile
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = (20, 6)

class arbre:

# à compléter
    def __init__(self):
        pass

    def taille(self, t):
        if self.estVide(t):
            return 0
        else:
            taille_gauche=self.taille(t[1])
            taille_droite=self.taille(t[2])
            return 1+taille_gauche+taille_droite

    def hauteur(self, t):
        if self.estVide(t):
            return -1
        else:
            hauteur_gauche=self.hauteur(t[1])
            hauteur_droite=self.hauteur(t[2])
            return 1+max(hauteur_gauche, hauteur_droite)

    def estVide(self, t):
        if t==[] or t==None:
            return True
        else:
            return False

    def dessiner_aux(self,t, rect, dy, labels, avl=False):
        if (self.estVide(t)):
            return
        x1, x2, y1, y2=rect
        xm= (x1+x2) //2
        x, t1, t2 = t
        self.dessiner_aux(t1, (x1, xm, y1, y2-dy), dy, labels, avl)
        self.dessiner_aux(t2, (xm, x2, y1, y2-dy), dy, labels, avl)
        if labels:
            if avl:
                plt.text(xm, y2, str(x[0]), fontsize=12, horizontalalignment='center')
            else:
                plt.text(xm, y2, str(x), fontsize=12, horizontalalignment='center')
        if not self.estVide(t1):
            a, b= ((xm, (x1+xm) //2), (y2, y2-dy))
            plt.plot(a, b, 'k', marker='o', markerfacecolor='c', markersize=25)
        if not self.estVide(t2):
            c, d= ((xm, (x2+xm) //2), (y2, y2-dy))
            plt.plot(c, d, 'k', marker='o', markerfacecolor='c', markersize=25)

    def dessiner(self,t, labels=True, avl=False):
        d=512
        pad=20
        dy= (d-2*pad) / (self.hauteur(t))
        self.dessiner_aux(t, (pad, d-pad, pad, d-pad), dy, labels, avl)
        plt.axis([0, d, 0, d])
        plt.axis('off')
        plt.show()

    def parcours_largeur(self,t):
        visite = PileFile()
        lst = []
        visite.enfiler(t)
        while visite.estVide() == False:
            visite.afficher()
            noeud = visite.defiler()
            if type(noeud) == list and noeud != []:
                lst.append(noeud[0])
                if noeud[1] != []:
                    visite.enfiler(noeud[1])
                if noeud[2] != []:
                    visite.enfiler(noeud[2])
            else:
                lst.append(noeud)
        return lst
```

```python
    def parcours_profondeur(self,t):
        visite = PileFile()
        lst = []
        visite.empiler(t)
        while visite.estVide() == False:
            visite.afficher()
            noeud = visite.depiler()
            if type(noeud) == list and noeud != []:
                lst.append(noeud[0])
                if noeud[2] != []:
                    visite.empiler(noeud[2])
                if noeud[1] != []:
                    visite.empiler(noeud[1])
            else:
                lst.append(noeud)
        return lst

    def parcours_profondeur_recursif_prefix(self, t):
        lst = []
        if type(t) == list and t != []:
            lst.append(t[0])
            print(lst)
            if t[1] != []:
                res1 = self.parcours_profondeur_recursif_prefix(t[1])
                lst.append(res1)
                print(lst)
            if t[2] != []:
                res2 = self.parcours_profondeur_recursif_prefix(t[2])
                lst.append(res2)
                print(lst)
        else:
            lst.append(t)
            print(lst)
        return lst

    def parcours_profondeur_recursif_infixe(self, t):
        lst = []
        if type(t) == list and t != []:
            if t[1] != []:
                res1 = self.parcours_profondeur_recursif_infixe(t[1])
                lst.append(res1)
                print(lst)
            lst.append(t[0])
            print(lst)
            if t[2] != []:
                res2 = self.parcours_profondeur_recursif_infixe(t[2])
                lst.append(res2)
                print(lst)
        else:
            lst.append(t)
            print(lst)
        return lst

    def parcours_profondeur_recursif_postfixe(self, t):
        lst = []
        if type(t) == list and t != []:
            if t[1] != []:
                res1 = self.parcours_profondeur_recursif_postfixe(t[1])
                lst.append(res1)
                print(lst)
            if t[2] != []:
                res2 = self.parcours_profondeur_recursif_postfixe(t[2])
                lst.append(res2)
                print(lst)
            lst.append(t[0])
        else:
            lst.append(t)
            print(lst)
        return lst
```